instituto superior de engenharia de lisboa

Departamento de Engenharia Informática

Licenciatura em Engenharia Informática e Multimédia

Processamento de Imagem e Visão

1º Semestre 2026/2027

Trabalho Prático nº1

Contagem e Classificação de Peças de Lego Retangulares

Docente: Tiago Gonçalves

8 de novembro de 2026

Turma: 52D

Trabalho realizado por:
Filipe Mendes: A48628
Mª Luísa Melo e Sampaio: A50799

Índice

1 Introdução **1**

2 Conjunto de Dados **3**

3 Leitura das Imagens **5**

4 Binarização **7**
4.1 Escolha do canal de cor 7
4.2 Limiar de Otsu 7
4.3 Limiar fixo 8
4.4 Implementação 9
4.5 Conclusões sobre a binarização 10

5 Melhoramento da Imagem **11**
5.1 Comentários e Problemas Identificados 13

6 Extração de Componentes Conexos **15**
6.1 Comentários e Problemas Identificados: 15

7 Extração de Propriedades e Classificação de Peças **17**

8 Apresentação dos Resultados Obtidos **21**
8.1 Contagem manual das peças 21
8.2 Resultados Obtidos 21

9 Conclusões **23**

Referências **25**

Lista de Figuras

1 Canais de cor da imagem lego09 7
2 Imagens com peças verdes e respetiva binarização pelo método de Otsu. 8
3 Histograma do canal vermelho da imagem lego21 9
4 Binarização da imagem lego21 com três limiares diferentes. 9
5 Melhoramento com elemento estruturante retangular e elíptico 11
6 Separação das peças encostadas na imagem lego04. 13
7 "Bico" na zona de contacto entre duas peças 14
8 Zona de contacto da imagem lego30 após o melhoramento, com erosão 15x15 e dois tamanhos de dilatação. 14
9 Resultado do método na listagem 4. 16
10 Imagem final classificada com listagem 6. 19

Lista de Tabelas

1 Situações difíceis presentes em cada imagem do conjunto de dados. . . 3
2 Efeito do número de iterações (erosão 15x15, dilatação 11x11) nas imagens de treino. 14
3 Contagem manual das peças de Lego. 21
4 Comparação entre a contagem manual e a do algoritmo 22

Lista de Listagens

1 Leitura de Imagens 5
2 Função correta de binarização 10
3 Função de melhoramento de imagem 13
4 Função de extração de componentes conexos 15
5 Função de classificação de peças LEGO por dimensões 17
6 Função de atribuição de labels para as peças LEGO 18
7 Codigo que guarda os resultados finais em imagem 20

1 Introdução

O presente relatório descreve o trabalho prático realizado no âmbito da unidade curricular de Processamento de Imagem e Visão, da Licenciatura em Engenharia Informática e Multimédia do ISEL.

Este trabalho propõe um algoritmo de visão por computador capaz de contar e classificar automaticamente peças de Lego retangulares de dimensões 2x2, 2x4, 2x6 e 2x8 num ambiente controlado. Esta solução tem em conta perturbações como a presença de outras peças de Lego ou objetos no campo de visão e eventual contacto entre as peças.

O desenvolvimento deste algoritmo visa consolidar os conhecimentos sobre a biblioteca *OpenCV* para programação de aplicações de visão por computador em tempo real para linguagem de programação *Python*.

O grupo começou por analisar o conjunto de dados onde fez a sua divisão em treino e validação e, definiu os critérios da contagem manual usada como referência. De seguida leu as imagens e comparou os três canais de cor escolhendo o vermelho por ser o que melhor separa as peças da mesa azul. Testou também o método de Otsu, que descartou por elimnar uma peça verde, optando assim por um limiar fixo. Depois, aplicou o fecho para preencher os buracos no interior das peças e erosões seguidas de dilatações para separar as peças encostadas, usando um elemento estruturante elíptico em vez de retangular,que desformava as peças que estavam na diagonal. Por fim, classificou as peças consoante as propriedades de cada uma delas.

2 Conjunto de Dados

As imagens foram divididas nos seguintes grupos:

- **Treino** (10 imagens): usadas para desenvolver o algoritmo e afinar os seus parâmetros. Foram escolhidas de forma a incluir pelo menos uma imagem de cada situação difícil presente no conjunto de dados: peças encostadas, uma peça sobreposta a outra, peças que não são retangulares, outros objetos e peças verdes, com pouco contraste no canal vermelho. Como os parâmetros só podem ser afinados para as situações que surgem durante o desenvolvimento, estas imagens tinham de pertencer ao treino.
- **Validação** (4 imagens): as restantes imagens, que não foram usadas na afinação dos parâmetros. Servem para estimar o desempenho do algoritmo em imagens que não viu durante o desenvolvimento.
- **Teste**: imagens fornecidas pelos docentes na discussão, adquiridas nas mesmas condições.

A Tabela 1 indica as situações difíceis em cada imagem.

Tabela 1: Situações difíceis presentes em cada imagem do conjunto de dados.

Conjunto	Imagem	Situações
Treino	lego01	Meia-lua, peça com o topo curvo, peças na diagonal
	lego02	Peça de lado, 2x4 encostada à 2x8 pelo canto
	lego04	Três peças encostadas em cadeia (2x2–2x4–2x2)
	lego09	2x6 encostada à 2x4 cinzenta pelo canto, peças na diagonal
	lego10	Minifigura encostada à 2x4, 2x4 encostada à 2x8
	lego11	Peça 2x2 sobreposta a uma 2x8
	lego12	Apenas peças inválidas: meia-lua, peça com o topo curvo e peça de lado
	lego13	Meia-lua encostada a uma 2x4
	lego21	Peça verde, com pouco contraste no canal vermelho
	lego30	Figura de animal, 2x2 encostada à 2x4 peça verde
Validação	lego22	Peça 2x2 rodada a 45°
	lego23	Peças 2x4 na diagonal
	lego25	—
	lego32	Figura de animal, peça cortada pelo limite da imagem

Como consequência desta divisão, nenhuma imagem de validação tem peças encostadas, pelo que o resultado obtido neste conjunto não avalia a separação de peças em contacto. No entanto, a imagem lego32 contém uma situação que não surge no treino, uma peça cortada pelo limite da imagem, que o algoritmo classificou corretamente como *Invalid*. Nenhuma das imagens apresenta sombras fortes, só sombras fracas junto às arestas das peças, pelo que não foi possível avaliar a robustez do algoritmo a esta perturbação.

Para avaliar o algoritmo, cada imagem foi contada manualmente, segundo os seguintes critérios:

- São consideradas peças válidas apenas as peças retangulares de dimensões 2x2, 2x4, 2x6 e 2x8 vistas de cima.
- Peças com faces curvas (meias-luas e peças 2x2 com o topo arredondado), peças pousadas de lado e outros objetos são contabilizados como *Invalid*.
- Peças parcialmente fora do campo de visão (lego32) são contabilizadas como *Invalid*, uma vez que não é possível determinar as suas dimensões.

A comparação entre esta contagem manual e os resultados do algoritmo é apresentada na Secção 8.

3 Leitura das Imagens

Para começar, são lidas todas as imagens do conjunto de treino. Os nomes dos ficheiros são guardados na lista `image_files` e as imagens lidas na lista `readImgs`. É importante realçar que a diretoria é definida manualmente, pelo que as imagens deste conjunto deverão estar todas dentro da pasta `./treino/`. Por fim, para que seja possível observar as imagens, faz-se o *plot* de todas as imagens de treino. Para fazer a validação, basta alterar a variável para `images_dir = './validacao/'`.

```
1  images_dir = './treino/' #'./validacao/' para as imagens de validacao
2
3  #Cria uma lista com os nomes dos ficheiros de imagem que terminam em '.jpg'
4  image_files = [f for f in os.listdir(images_dir) if f.endswith('.jpg')]
5
6  readImgs = [] #Lista de imagems originais em BGR
7  rgbImgs = [] #Lista de imagems em formato RGB
8
9  x = 1
10 plt.figure(figsize=(25,20))
11
12 #Percorre a lista de imagens originais no ficheiro onde estão guardadas
13 for i in range(len(image_files)):
14     image_path = os.path.join(images_dir, image_files[i])
15     image = cv2.imread(image_path)
16     readImgs.append(image)
17
18     #Converte a imagem de BGR para RGB
19     rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
20     rgbImgs.append(rgb_image)
21
22     #Exibição das imagens em formato RGB
23     plt.subplot(4, 4, x)
24     plt.imshow(rgbImgs[i])
25     plt.axis('off')
26     x += 1
```

Listagem 1: Leitura de Imagens

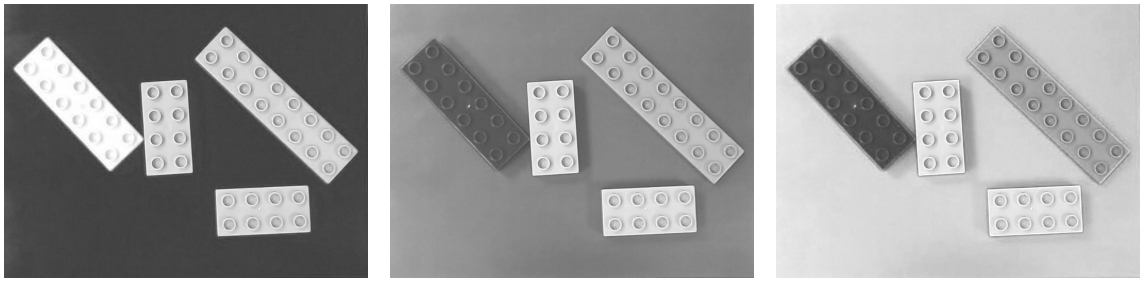
4 Binarização

Tal como o nome indica, a binarização trata o processo de divisão de dados em 2 grupos diferentes. Atribui-se o valor 0 a todos os valores abaixo do limiar de divisão, e 255 a todos os valores acima (Abrantes, 2026a).

O resultado, neste caso, será uma imagem com 2 cores: o fundo a preto, e os objetos a branco. Esta separação, ainda que por agora não distingue as peças de lego dos diferentes objetos, é essencial para separar o fundo dos restantes elementos.

4.1 Escolha do canal de cor

Para fazer a binarização, era preciso escolher qual dos canais de cor seria melhor para a visualização e melhor aplicação do que é necessário para este trabalho. Para isso, as imagem foram analisadas, e concluiu-se que o canal vermelho seria o mais indicado, devido ao azul que é usado sempre na mesa em qual as fotos de teste foram feitas.



(a) Canal vermelho **(b)** Canal verde **(c)** Canal azul

Figura 1: Canais de cor da imagem lego09

Como é possível ver na figura 1, o canal vermelho é o mais adequado para a separação da mesa e das peças.

Neste canal, a mesa, por ser azul, reflete pouca luz vermelha e aparece escura, enquanto as peças aparecem claras. Nos canais verde e azul, a mesa tem valores elevados, próximos dos das peças, pelo que o contraste entre ambas é menor.

4.2 Limiar de Otsu

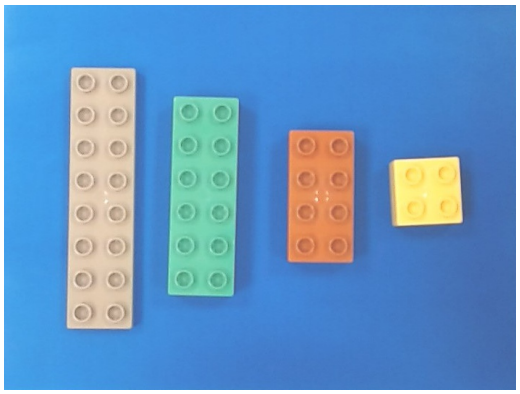
Quando são usados limiares fixos, o valor desses limiares tem de ser especificado manualmente para cada cenário de iluminação ou contraste. O **Método de Otsu**, disponibilizado pela biblioteca *OpenCV* escolhe o limiar automaticamente a partir do histograma da imagem (GeeksforGeeks, 2025b).

Para definir a melhor separação, o algoritmo de Otsu procura um limiar que torne os dois grupos (fundo e objeto) o mais distintos possível entre si (Abrantes, 2026a). Matematicamente, isto pode ser alcançado minimizando a variância interna de cada classe (σ_W^2), garantindo que os píxeis do mesmo grupo sejam homogéneos, ou maximizando a variância entre as médias das duas classes (σ_B^2), garantindo o máximo de contraste entre elas. Como a variância total da imagem é constante ($\sigma^2 = \sigma_W^2 + \sigma_B^2$), maximizar a variância inter-classes (σ_B^2) conduz exatamente ao mesmo limiar ótimo e requer um esforço computacional menor (Abrantes, 2026a).

Por depender exclusivamente do histograma global, o método de Otsu é sensível a ruído, pelo que é habitual aplicar previamente um desfoque gaussiano (`cv2.GaussianBlur`), que suaviza o valor dos píxeis e reduz irregularidades antes da binarização (OpenCV Team, 2024c).

O grupo testou este método, mas verificou que, em algumas imagens, peças inteiras eram eliminadas. Nas imagens lego21 e lego30 (Figura 2), a peça verde desaparece por completo com o método de Otsu. Isto deve-se porque um objeto verde reflete sobretudo luz verde e pouca luz vermelha, pelo que esta peça apresente no canal vermelho, valores intermédios entre a mesa e as restantes peças.

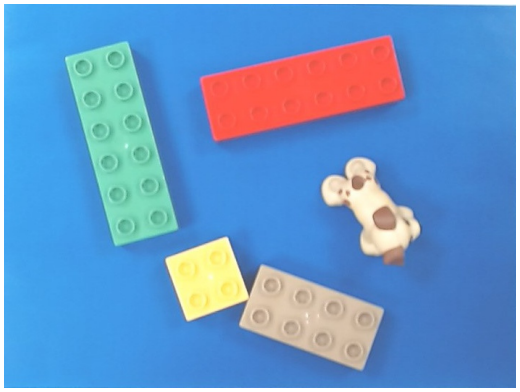
Ao separar a imagem em apenas duas classes, o método de Otsu escolhe um limiar que acaba por agrupar a peça verde com o fundo.



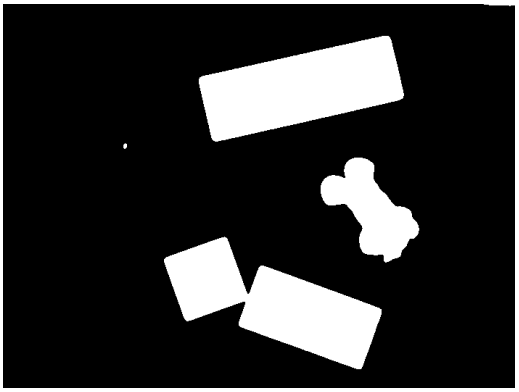
(a) lego21, original



(b) lego21, Otsu



(c) lego30, original



(d) lego30, Otsu

Figura 2: Imagens com peças verdes e respetiva binarização pelo método de Otsu.

4.3 Limiar fixo

Como alternativa ao método de Otsu, testaram-se limiares fixos na imagem lego21. O histograma do canal vermelho desta imagem (Figura 3) apresenta três picos: o da mesa, em torno do valor 80, o da peça verde, em torno de 120, e o das restantes peças, em torno de 230. Para que a peça verde não seja eliminada, o limiar tem de ficar entre os dois primeiros picos. O valor 85 situa-se no vale que os separa, onde há poucos píxeis, pelo que pequenas variações do limiar alteram pouco o resultado.

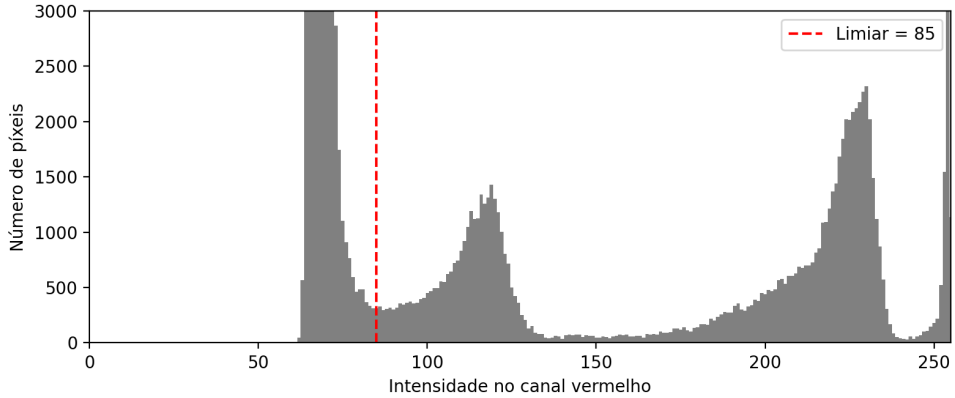
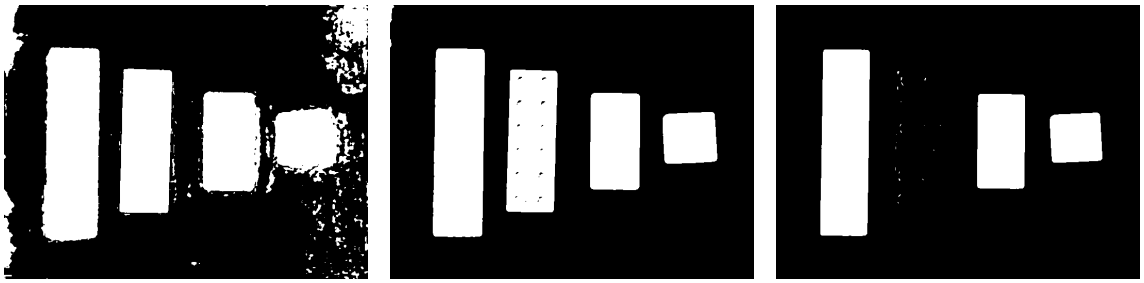


Figura 3: Histograma do canal vermelho da imagem lego21 após desfoque gaussiano. O eixo vertical foi limitado a 3000 píxeis; o pico da mesa atinge cerca de 67 000.

A Figura 4 compara três limiares na mesma imagem. Com 70, parte da mesa é considerada objeto. Com 130, a peça verde desaparece quase por completo. Com 85, as peças ficam separadas do fundo, embora a peça verde apresente pequenos buracos, que são preenchidos pelo fecho descrito na secção seguinte.



(a) Limiar 70 **(b)** Limiar 85 **(c)** Limiar 130

Figura 4: Binarização da imagem lego21 com três limiares diferentes.

4.4 Implementação

Foram feitas duas funções para realizar a binarização:

- `binarization` é uma versão original da função que retinha erros, especificamente onde perdia algumas das peças pois estas eram bastante proximas da cor da mesa.
- `bin_red_chanel` é a função usada neste trabalho para binarizar as imagens. Começa por extrair o canal vermelho da imagem e aplica-lhe um desfoque gaussiano. De seguida, binariza a imagem com `cv2.threshold` e `cv2.THRESH_BINARY`, que atribui o valor 255 aos píxeis acima do limiar de 85 e 0 aos restantes, devolvendo a imagem binarizada. O canal vermelho corresponde ao índice 2, porque o OpenCV lê as imagens no formato BGR. O desfoque gaussiano é aplicado para reduzir o ruído antes da binarização, pelo mesmo motivo referido para o método de Otsu (Abrantes, 2026c).

```python
#Função para binarizar as imagens através de um threshold no canal vermelho
def bin_red_chanel(img):
    #Aplica um desfoque gaussiano ao canal vermelho da imagem
    aux = cv2.GaussianBlur(img[:,:,2], (5,5), 3)
    #Aplica um threshold no canal vermelho da imagem RGB
    _, imgBinarizada = cv2.threshold(aux, 85, 255, cv2.THRESH_BINARY)

    return imgBinarizada
```

Listagem 2: Função correta de binarização

4.5 Conclusões sobre a binarização

A comparação dos três canais de cor mostrou que o canal vermelho é o que melhor separa as peças da mesa azul. O método de Otsu foi rejeitado porque, ao dividir o histograma em apenas duas classes, agrupa a peça verde com o fundo nas imagens lego21 e lego30. Optou-se então por um limiar fixo de 85, que, entre os valores testados, separa todas as peças do fundo sem incluir a mesa. Antes da binarização, é aplicado um desfoque gaussiano para reduzir o ruído.

5 Melhoramento da Imagem

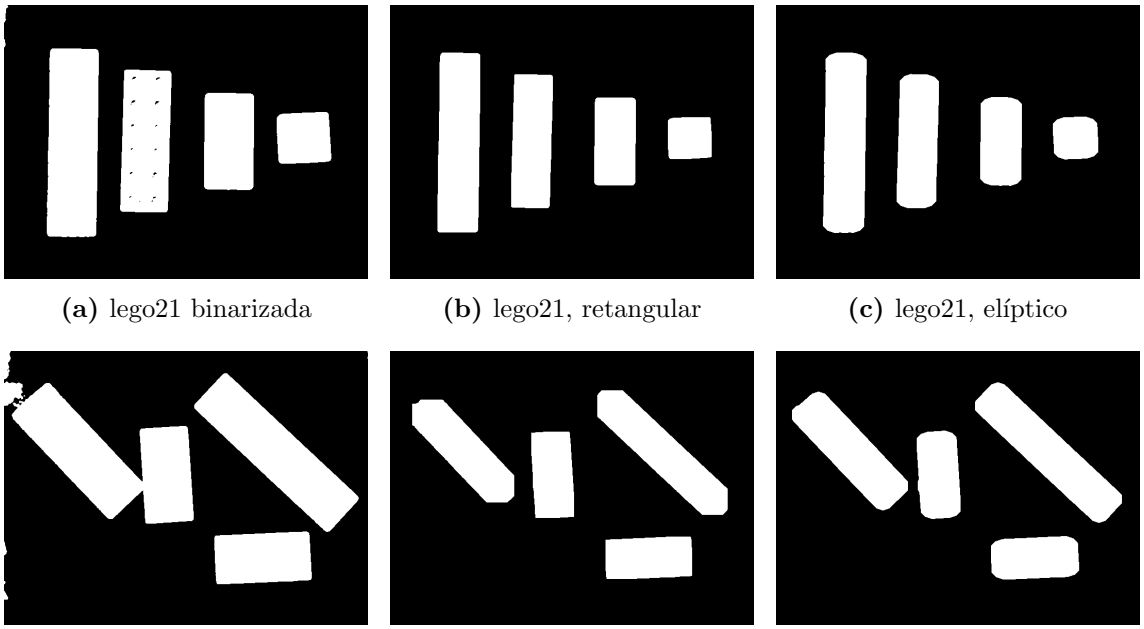
Apesar de o canal vermelho ter isolado a maior parte das peças da mesa, o resultado da binarização ainda apresenta problemas: existem pequenos buracos no interior das peças mais escuras (como a verde) e as peças que estão encostadas umas às outras aparecem unidas como se fossem um único objeto. Procura-se agora resolver estes problemas através de operações morfológicas da biblioteca *OpenCV*.

Podem ser usadas para abrir ou fechar buracos que possam surgir nos objetos. Operam nas imagens através de um elemento estruturante, que define a vizinhança a ser examinada por cada píxel (Abrantes, 2026a; GeeksforGeeks, 2025c).

O operador `cv2.MORPH_CLOSE` executa uma dilatação seguida de uma erosão. É por isso útil para fechar os pequenos pontos pretos que temos nos objetos, e que deveriam ficar completamente brancos (OpenCV Team, 2024d).

Outro ponto importante é que algumas peças de lego ficam encostadas e com os seus limites sobrepostos. Para resolver isso, efetuam-se erosões até se conseguir separar as duas componentes. Para manter os tamanhos mais próximos dos originais, aplicam-se dilatações até estas ficarem semelhantes, usando um elemento estruturante menor.

Visto que se tratam de objetos retangulares, foi inicialmente usado um elemento estruturante retangular. Porém foram observadas incoerências nas peças que estão nas diagonais:



(a) lego21 binarizada **(b)** lego21, retangular **(c)** lego21, elíptico

(d) lego09 binarizada **(e)** lego09, retangular **(f)** lego09, elíptico

Figura 5: Melhoramento com elemento estruturante retangular e elíptico. Na primeira linha, peças verticais (lego21); na segunda, peças na diagonal (lego09). À esquerda, a imagem binarizada antes do melhoramento.

Na Figura 5, com o elemento retangular, as peças verticais da lego21 mantêm os cantos (b), enquanto com o elíptico ficam com as extremidades arredondadas (c). Nas peças na diagonal da lego09, o elemento retangular deforma as extremida-

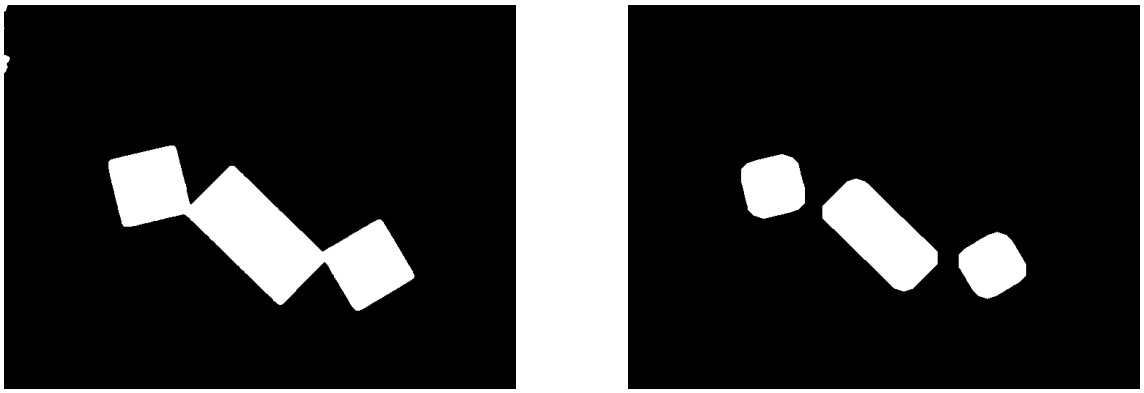
des (e), o que pode comprometer a classificação, enquanto o elíptico arredonda as extremidades de forma igual em qualquer orientação (f).

```python
#Função para melhorar a imagem binarizada
def get_better_image(img):
    #Operação morfológica de fechamento para preencher buracos na imagem
    closed  = cv2.morphologyEx(img, cv2.MORPH_CLOSE, cv2.getStructuringElement (cv2.MORPH_ELLIPSE, (5, 5)))
    #Erosão para remover ruído; utiliza um elemento estruturante elíptico de 15x15 e 3 iterações
    eroded = cv2.erode(closed, cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (15, 15)), iterations=3)
    #Dilatação para restaurar a forma original da imagem após a erosão; utiliza um elemento estruturante elíptico de 11x11 e 4 iterações
    dilated = cv2.dilate(eroded, cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (11,11)), iterations=3)
    return dilated  # Retorna a imagem melhorada
```

Listagem 3: Função de melhoramento de imagem

5.1 Comentários e Problemas Identificados

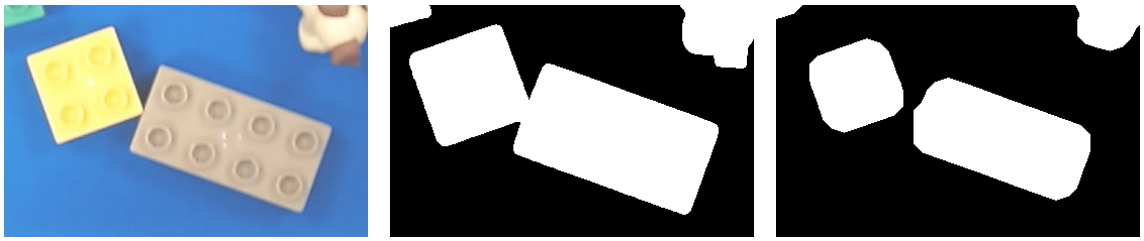
Após o melhoramento, as peças que se encontravam encostadas passam a estar separadas. Na imagem lego04 (Figura 6), as três peças formam um único objeto após a binarização (a) e três objetos distintos após o melhoramento (b). Os parâmetros do tamanho dos elementos estruturantes foram testados até obter os resultados agora apresentados, que foram considerados como sendo os melhores.



(a) Após a binarização **(b)** Após o melhoramento

Figura 6: Separação das peças encostadas na imagem lego04.

Um problema que surge agora, é que após esta separação, as peças apresentam pequenos "bicos" dos lados que antes estavam encostados, deixando por isso de ser retângulos perfeitos. Isto será relativamente importante mais à frente quando o classificador for implementado.



(a) Original **(b)** Após a binarização **(c)** Após o melhoramento

Figura 7: Formação de um "bico" na zona de contacto entre duas peças da imagem lego30.

Os parâmetros do melhoramento determinam quanto as peças são reduzidas e depois recuperadas. A erosão tem de remover a zona de contacto entre peças encostadas, mas não pode ser tão forte que elimine objetos finos ou peças pequenas. A dilatação usa um elemento menor do que a erosão (11x11 contra 15x15) porque, com o mesmo tamanho, as peças separadas voltavam a crescer até se tocarem: nos testes, isto aconteceu em cinco das dez imagens de treino (lego02, lego09, lego10, lego13 e lego30). A Figura 8 mostra este efeito na imagem lego09.



(a) Dilatação 11x11 (utilizada) **(b)** Dilatação 15x15 (igual à erosão)

Figura 8: Zona de contacto da imagem lego30 após o melhoramento, com erosão 15x15 e dois tamanhos de dilatação.

A Tabela 2 mostra o efeito do número de iterações nas imagens de treino. Com uma ou duas iterações, várias peças encostadas não são separadas. Com seis, a erosão deforma ou elimina peças. Os melhores resultados obtêm-se entre três e cinco iterações.

Tabela 2: Efeito do número de iterações (erosão 15x15, dilatação 11x11) nas imagens de treino.

Iterações	Acertos	Imagens com erros
1	32/45	7 imagens
2	36/45	6 imagens
3	44/45	lego11
4	43/45	lego10, lego11
5	43/45	lego10, lego11
6	40/45	5 imagens

6 Extração de Componentes Conexos

A extração de componentes conexos identifica cada objeto da imagem binária e obtém o seu contorno. Este passo é aplicado a todos os objetos, sejam ou não peças.

Para tal, foi utilizado o método `cv2.findContours` da biblioteca OpenCV (GeeksforGeeks, 2025a; OpenCV Team, 2024a). Cada componente conexo tem um contorno exterior e, se tiver buracos, um contorno interior por cada buraco. O método permite escolher que contornos são devolvidos:

- `cv2.RETR_EXTERNAL`: devolve apenas os contornos exteriores;
- `cv2.RETR_CCOMP`: devolve os contornos exteriores e interiores, organizados numa hierarquia de dois níveis;
- `cv2.RETR_TREE`: devolve os contornos exteriores e interiores, organizados numa hierarquia completa, em árvore;
- `cv2.RETR_LIST`: devolve todos os contornos, sem estabelecer relação entre eles.

A função de classificação trata cada contorno devolvido como um objeto e não usa a hierarquia, pelo que um contorno interior seria contado como uma peça adicional. Optou-se, por isso, pelo modo `cv2.RETR_EXTERNAL`. Nas imagens deste trabalho, o fecho preenche os buracos das peças, pelo que `cv2.RETR_EXTERNAL` e `cv2.RETR_TREE` devolvem os mesmos contornos; a escolha torna-se relevante caso um objeto mantenha um buraco após o melhoramento. O parâmetro `cv2.CHAIN_APPROX_SIMPLE` comprime os segmentos horizontais, verticais e diagonais do contorno, guardando apenas as suas extremidades, o que reduz a memória ocupada sem alterar a forma do contorno (OpenCV Team, 2024b). O resultado pode ser visto na Figura 9.

```python
#Função para encontrar contornos nas imagens melhoradas
def find_contours(better_image):
    #Encontra os contornos da imagem usando cv2.findContours
    #RETR_EXTERNAL devolve apenas os contornos exteriores; CHAIN_APPROX_SIMPLE reduz o número de pontos
    contours, hierarchy = cv2.findContours(better_image, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    return contours, hierarchy
```

Listagem 4: Função de extração de componentes conexos

6.1 Comentários e Problemas Identificados:

Observando os objetos detetados para cada imagem, verifica-se uma contagem conforme a esperada.

Os contornos ficam desenhados "internamente" em relação aos verdadeiros limites dos objetos. Isto acontece porque a dilatação usa um elemento estruturante mais pequeno do que a erosão. Esta redução altera o *aspect ratio* das peças alongadas mas não das peças quadradas. Este efeito não compromete a classificação porque os valores definidos na função foram definidos a partir das peças depois do melhoramento.

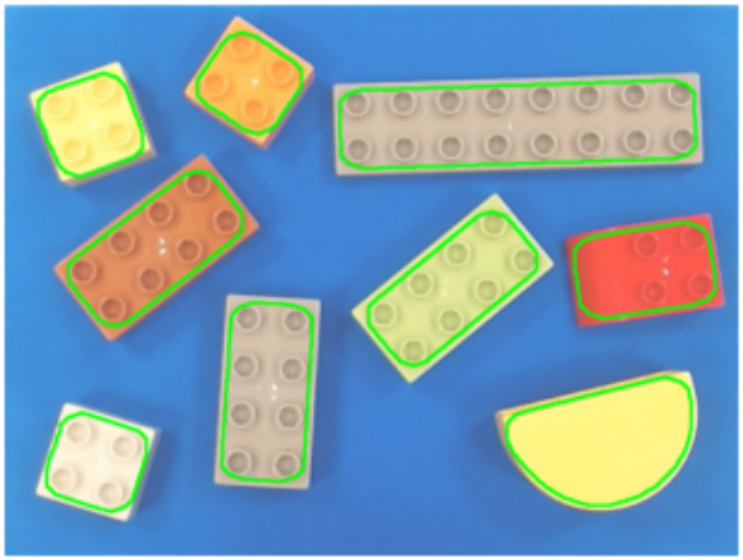


Figura 9: Resultado do método na listagem 4.

7 Extração de Propriedades e Classificação de Peças

A classificação das peças é feita pela função `classify_lego_piece` (Listagem 5), que atribui a cada objeto uma das classes 2x2, 2x4, 2x6 ou 2x8, ou o classifica como inválido.

```python
#Função para classificar uma peça de LEGO com base nas suas dimensões
def classify_lego_piece(w, h):
    #Calcula o aspect ratio com base na largura e altura da peça
    aspect_ratio = max(w, h) / min(w, h)

    if w < 60 or h < 60:
        return "Invalid"

    if aspect_ratio > 1.2:
        return "2x2"

    if 1.8 <= aspect_ratio <= 2.4:
        return "2x4"

    if 2.4 <= aspect_ratio < 3.5:
        return "2x6"

    if aspect_ratio > 3.5:
        return "2x8"

    return "Invalid"
```

Listagem 5: Função de classificação de peças LEGO por dimensões

O método recebe os parametros w e h do retângulo para calcular o *aspect ratio* (OpenCV Team, 2025) (`aspect_ratio = max(w, h) / min (w, h)`), pelo que é sempre igual ou superior a 1 e não depende da orientação da peça. Para uma peça 2xn, este valor aproxima-se de $n/2$: cerca de 1 numa 2x2, 2 numa 2x4, 3 numa 2x6 e 4 numa 2x8. Os valores medidos nas imagens são ligeiramente superiores, devido à redução das peças no melhoramento.

Objetos com um dos lados inferior a 60 píxeis são considerados inválidos, por serem demasiado pequenos para uma peça. Os restantes são classificados de acordo com o intervalo em que se encontra o seu *aspect ratio*. Os valores entre 1,25 e 1,82 não correspondem a nenhuma peça e são também classificados como inválidos, o que permite rejeitar objetos como as meias-luas.

Por fim, o método abaixo na listagem 6 demonstrado é chamado em `classify_and_label_components`, que recebe os contornos obtidos correspondentes à imagem original, assim como as suas dimensões, e atribui um label a cada peça correspondente:

```
def classify_and_label_components(orig_image, contours):
    #Contadores para cada tipo de peça de LEGO e para peças inválidas
    x2Count = 0
    x4Count = 0
    x6Count = 0
    x8Count = 0
    invalidCount = 0

    for contour in contours:
        area = cv2.contourArea(contour)  #Calcula a área do contorno
        if area < 1000:  #Ignora contornos com área inferior a 1000
            continue

        rect = cv2.minAreaRect(contour)  #Obtém um retângulo mínimo que envolve o contorno
        (x, y), (w, h), angle = rect  #Extrai as coordenadas e dimensões do retângulo
        w, h = sorted([w, h])  #Ordena a largura e altura para estas não serem trocadas a calcular o aspect ratio

        #Aproxima o contorno para uma forma poligonal
        epsilon = 0.02 * cv2.arcLength(contour, True)
        approx = cv2.approxPolyDP(contour, epsilon, True)

        if len(approx) != 0:
            #Classifica a peça de LEGO com base na largura e altura
            label = classify_lego_piece(w, h)

            # Atualiza o contador com base na classificação
            if label == "2x2":
                x2Count += 1
            elif label == "2x4":
                x4Count += 1
            elif label == "2x6":
                x6Count += 1
            elif label == "2x8":
                x8Count += 1
            else:
                label = "Invalid"
                invalidCount += 1

            #Desenha o contorno na imagem original
            cv2.drawContours(orig_image, [contour], -1, (0, 255, 0), 2)

            #Adiciona o label da peça na imagem
            cv2.putText(orig_image, label, (int(x), int(y) - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)

    #Cria uma faixa preta por baixo da imagem para escrever a contagem sem tapar as peças
    faixa = np.zeros((40, orig_image.shape[1], 3), dtype=np.uint8)
    cv2.putText(faixa, f"2x2: {x2Count}   2x4: {x4Count}   2x6: {x6Count}   2x8: {x8Count}   Invalidos: {invalidCount}",
                (10, 27), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 1)
    #Junta a faixa à imagem original
```

```
50      orig_image = np.vstack([orig_image, faixa])
51
52      return orig_image
```

Listagem 6: Função de atribuição de labels para as peças LEGO

Este método final, começa por calcular a área do contorno usando `cv2.contourArea`, e ignora formas que forem demasiado pequenas para serem classificadas.

Passa depois para obter as coordenadas de cada peça, colocando um retângulo à volta de cada contorno e extraindo as suas coordenadas e dimensões para poder colocar os *labels* em cima das peças correspondentes.

De seguida, é feita uma verificação para contornos como arcos usando `cv2.approxPolyDP`, um contorno que seja determinado como curvo, tem o seu numero de pontos reduzido para poder ser feita a verificação das dimensões com `classify_lego_piece`. Isto resulta com as peças com arcos tenham um *aspect ratio* que não as permita serem consideradas peças válidas.

Foi também implementado um filtro condicional para ignorar contornos com área inferior a 1000 píxeis, eliminando assim pequenos artefactos ou ruído gerado por reflexos que pudessem ser contabilizados erradamente como objetos.

O resto do método, faz uma contagem simples do número de peças de cada tipo para as apresentar no canto superior esquerdo usando `cv2.putText`, e desenha os contornos em cima da imagem original através da função `cv2.drawContours`. No fim obtem-se uma imagem que mostra corretamente toda a informação necessária sobre as peças apresentadas: contornos, dimensões e número de peças de cada tipo:

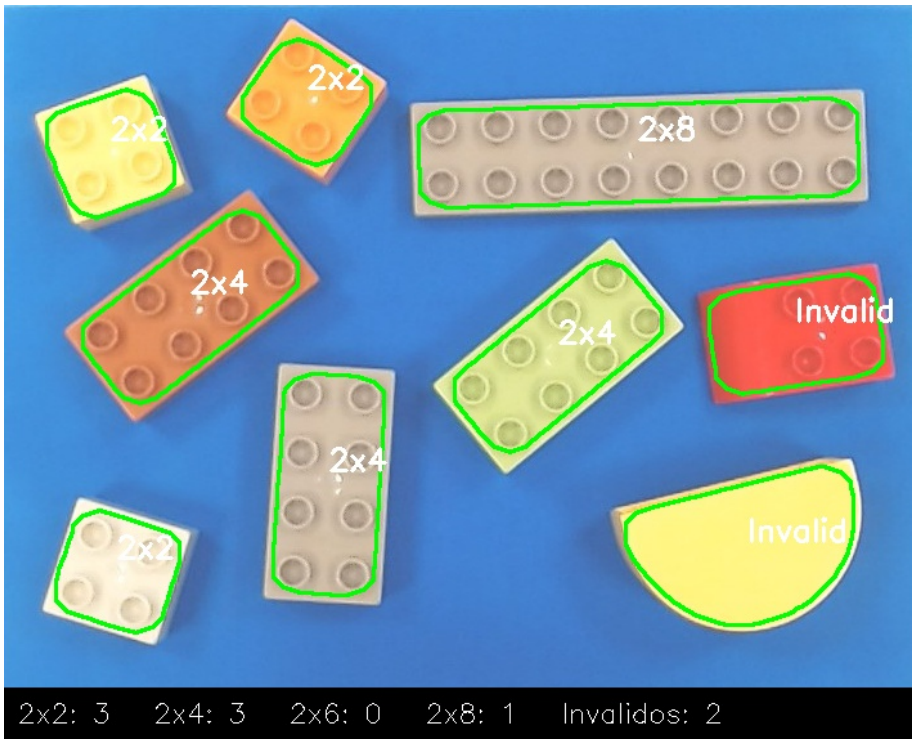


Figura 10: Imagem final classificada com listagem 6.

Por fim, estes resultados são guardados em imagens, dentro de uma pasta chamada `output_images` que ao dar *run* é criada automaticamente, visto abaixo na

listagem 7

```
folder_name = "output_images"  #Nome da pasta onde as imagens serão guardadas

#Verifica se a pasta já existe; se não, cria-a
if not os.path.exists(folder_name):
    os.makedirs(folder_name)

for i, image in enumerate(classImgs):
    #Define o path do arquivo onde a imagem será guardada, nomeando-a com base no indice
    image_path = os.path.join(folder_name, f"image_{i}.jpg")
    cv2.imwrite(image_path, image)
```

Listagem 7: Codigo que guarda os resultados finais em imagem

8 Apresentação dos Resultados Obtidos

8.1 Contagem manual das peças

A Tabela 3 contém a classificação de peças e contagem manual de cada imagem. Esta tabela vai servir posteriormente para a comparação com os resultados obtidos.

Tabela 3: Contagem manual das peças de Lego.

Conjunto	Imagem	2x2	2x4	2x6	2x8	Inválidas	Total
Treino	lego01	3	3	0	1	2	9
	lego02	2	2	0	1	1	6
	lego04	2	1	0	0	0	3
	lego09	0	2	1	1	0	4
	lego10	0	1	0	1	1	3
	lego11	1	2	0	1	0	4
	lego12	0	0	0	0	3	3
	lego13	0	2	1	0	1	4
	lego21	1	1	1	1	0	4
	lego30	1	1	2	0	1	5
	Subtotal	10	15	5	6	9	45
Validação	lego22	4	0	0	0	0	4
	lego23	0	4	0	0	0	4
	lego25	0	0	0	1	0	1
	lego32	1	1	1	0	2	5
	Subtotal	5	5	1	1	2	14
Total		15	20	6	7	11	59

8.2 Resultados Obtidos

A Tabela 4 compara, para cada imagem, a contagem manual (real) com a contagem obtida pelo algoritmo.

Tabela 4: Comparação entre a contagem manual e a contagem do algoritmo. Nas colunas das classes, real/obtido; na coluna Acertos, objetos corretos/total. Os erros estão assinalados a vermelho.

Conjunto	Imagem	2x2	2x4	2x6	2x8	Inválidos	Acertos
Treino	lego01	3/3	3/3	0/0	1/1	2/2	9/9
	lego02	2/2	2/2	0/0	1/1	1/1	6/6
	lego04	2/2	1/1	0/0	0/0	0/0	3/3
	lego09	0/0	2/2	1/1	1/1	0/0	4/4
	lego10	0/0	1/1	0/0	1/1	1/1	3/3
	lego11	1/0	2/2	0/0	1/1	0/0	3/4
	lego12	0/0	0/0	0/0	0/0	3/3	3/3
	lego13	0/0	2/2	1/1	0/0	1/1	4/4
	lego21	1/1	1/1	1/1	1/1	0/0	4/4
	lego30	1/1	1/1	2/2	0/0	1/1	5/5
	Subtotal	10/9	15/15	5/5	6/6	9/9	44/45
Validação	lego22	4/4	0/0	0/0	0/0	0/0	4/4
	lego23	0/0	4/4	0/0	0/0	0/0	4/4
	lego25	0/0	0/0	0/0	1/1	0/0	1/1
	lego32	1/1	1/2	1/1	0/0	2/1	4/5
	Subtotal	5/5	5/6	1/1	1/1	2/1	13/14

Analisando os **subtotais**, no conjunto de treino, o algoritmo acertou em **97,8%** dos casos (44 em 45 objetos corretos), falhando apenas numa peça retangular. No conjunto de validação o algoritmo acertou em 13 dos 14 objetos (**92,9%**). Como os parâmetros foram afinados com as imagens de treino, o resultado da validação é o que melhor estima o desempenho do algoritmo em imagens novas (Abrantes, 2026b).

Verificou-se 1 erro em cada conjunto. Na imagem lego11 (treino) existe uma peça vermelha 2x2 por cima de uma peça 2x8, que acabaram por serem contabilizadas como uma peça 2x8. Na imagem lego32 (validação), a figura animal é calssificada comoo 2x4 porque o seu *aspect ratio* está dentro do intervalo definido para este tamanho de peça.

Relativamente às perturbações indicadas no enunciado, as peças encostadas foram corretamente separadas pelas operações morfológicas e todas as que não são peças retangulares foram classificadas como inválidas

Embora o algoritmo não resolva esta situação porque a silhueta exterior forma um único componente conexo, uma possível solução no futuro passaria por aplicar um detetor de arestas (como o filtro de Canny) associado ao algoritmo de *Watershed*. Isto permitiria detetar as linhas interiores que dividem o relevo das duas peças e forçar a separação dos dois objetos mesmo quando sobrepostos. Uma outra solução possível seria isolar a cor da peça vermelha que está por cima mas depois ficaria com um buraco na peça de baixo que poderia trazer mais complicações.

9 Conclusões

Na primeira fase do trabalho, o grupo dividiu as imagens fornecidas em dois conjuntos, um de treino e outro de validação, e contou manualmente as peças de cada imagem, de forma a ter uma referência para avaliar o algoritmo.

De seguida, fez-se a binarização das imagens. Depois de comparar os três canais de cor, o grupo concluiu que o canal vermelho era o que melhor separava as peças da mesa azul. O método de Otsu foi testado e rejeitado, por eliminar a peça verde das imagens lego21 e lego30. Em alternativa, utilizou-se um limiar fixo de 85, escolhido a partir do histograma do canal vermelho por se situar no vale entre os valores da mesa e os da peça verde.

O grupo passou então ao melhoramento das imagens binarizadas, recorrendo a operações morfológicas. O fecho permitiu preencher os pequenos buracos no interior das peças, e a combinação de erosões e dilatações permitiu separar as peças que se encontravam encostadas. Nesta fase, verificou-se ainda que o elemento estruturante elíptico preservava melhor a forma das peças na diagonal do que o retangular. Isto acontece porque o elemento elíptico arredonda as pontas de forma uniforme independentemente da rotação da peça, enquanto o elemento retangular deforma os cantos das peças rodadas, o que prejudicaria o cálculo do *aspect ratio* na fase de classificação.

Na extração de componentes conexos, foram obtidos os contornos de cada objeto com a função `findContours`. Por fim, na classificação, foi calculado o *aspect ratio* do retângulo mínimo de cada contorno, o que permitiu distinguir as peças 2x2, 2x4, 2x6 e 2x8 e identificar como inválidos os restantes objetos.

O algoritmo desenvolvido contou e classificou corretamente 58 dos 59 objetos presentes nas imagens, incluindo todos os objetos do conjunto de validação, e mostrou-se robusto à presença de outros objetos e ao contacto entre peças. A principal limitação identificada é a impossibilidade de distinguir peças sobrepostas, que formam um único componente conexo..

Em suma, acredita-se ter abordado os pontos essenciais do enunciado, demonstrando assim o conhecimento adquirido sobre o tema. Adicionalmente, este trabalho permitiu ao grupo consolidar os conceitos abordados nas aulas e aprofundar significativamente os conhecimentos sobre a biblioteca OpenCV e o processamento de imagem.

Referências

Abrantes, A. (2026a). Análise de imagens binárias. *Instituto Superior de Engenharia de Lisboa*.

Abrantes, A. (2026b). Conceitos de reconhecimento de padrões. *Instituto Superior de Engenharia de Lisboa*.

Abrantes, A. (2026c). Pré-processamento. *Instituto Superior de Engenharia de Lisboa*.

GeeksforGeeks. (2025a). Find and draw contours using OpenCV — Python. https://www.geeksforgeeks.org/python/find-and-draw-contours-using-opencv-python/

GeeksforGeeks. (2025b). Optimum global thresholding using Otsu's method. https://www.geeksforgeeks.org/computer-vision/optimum-global-thresholding-using-otsus-method/

GeeksforGeeks. (2025c). Python OpenCV – Morphological operations. https://www.geeksforgeeks.org/python/python-opencv-morphological-operations/

OpenCV Team. (2024a). Contours hierarchy. https://docs.opencv.org/4.x/d9/d8b/tutorial_py_contours_hierarchy.html

OpenCV Team. (2024b). Contours: Getting started. https://docs.opencv.org/3.4/d4/d73/tutorial_py_contours_begin.html

OpenCV Team. (2024c). Image thresholding. https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html

OpenCV Team. (2024d). Morphological transformations. https://docs.opencv.org/3.4.20/d9/d61/tutorial_py_morphological_ops.html

OpenCV Team. (2025). Contour properties. https://docs.opencv.org/4.x/d1/d32/tutorial_py_contour_properties.html